

JavaScript

一种运行在客户端(浏览器)的编程语言, 实现人机交互效果

作用: 网页特效; 表单合法性验证; 数据交互(获取后端数据渲染到前端); 服务端编程(node.js)

特点:

① 脚本编写语言

由ASCII码构成, 可直接用任何文本编辑器开发完成.

不必预先编译, 只要利用适当的解释器即可执行的解释式程序

注释: // /* */

② 基于对象的语言

既可以创建对象, 又可以使用现有的对象

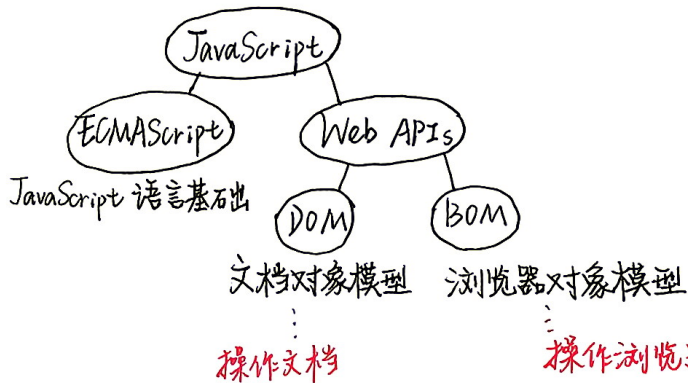
③ 动态性

是事件驱动的, 无需经过Web服务器就可以响应用户的输入

④ 跨平台

依赖于浏览器本身而与操作环境无关

组成:



(移动页面元素、调节大小、增删元素等) (页面弹窗、检测窗口宽度、存储数据到浏览器等)

书写位置有3种:

① 行内式: 在标签中嵌入 JavaScript 代码

`<p onmouseover = "alert('欢迎!')" > 鼠标移过来会弹窗哦 </p>`

② 内嵌式: JavaScript 代码放入 `<script>...</script>` 标记符中

```
<script language = "JavaScript">
    document.write("欢迎!");
</script>
```

③ 外部文件: 将代码独立存储为以.js为扩展名的文件, 利用SRC属性将该文件调入

```
<script language = "JavaScript" src = "welcome.js" > </script>
```

welcome.js:

```
document.write("欢迎!");
```

每条语句最后的分号(代码结束符)可以省略!

输入输出

① `document.write("要输出的内容");`

向body中输出内容，若内容中含有html标签，标签也会被解析出来

② `alert("要输出的内容");`

页面弹出警告对话框

③ `console.log("控制台打印");`

向控制台打印内容

④ `prompt("请输入你的姓名");`

显示对话框以提示用户输入文字

定义变量

`let age = 18;`
`let name = "小羊";`
`let score = 300.0;`
`age = '小杨';`

→ 数字类型

let 变量名 = 值

→ 字符串类型

JavaScript 是动态类型（变量类型由赋值的值决定）且弱类型（允许隐式类型转换，数字和字符串可直接拼接）的语言，因此不写数据类型

简单数据类型

① 数值型 (Number)：包括整点数和浮点数

② 布尔型 (Boolean)：true 或 false

③ 字符串 (String)：用单引号或双引号括起来的零个或多个字符或数字组成

undefined：定义了变量但没有给值

数据类型转换

① 自动数据类型转换

若表达式中使用 "+" 运算符, 且其中一个操作数是字符串而另一个操作数是数值时。

JavaScript 自动将数值转成字符串。

```
let x = "我今年" + 18;
```

结果 x = "我今年18"

```
let x = "15" + 8;
```

结果 x = 158

```
let x = 15 + 8;
```

结果 x = 23

若表达式中使用其它运算符, JavaScript 自动将字符串转成数值

```
let x = "30" / 5;
```

结果 x = 6

```
let x = "15" - "8";
```

结果 y = 7

② 数据类型转换函数

转为 "字符串" 类型

```
let x = 10;
```

```
console.log(x.toString());
```

```
console.log(String(x)); // 强制类型转换
```

将字符串转换成整数数值型

```
let x = parseInt("129");
```

```
console.log(a);
```

或 let x = Number("129");

```
console.log(a);
```

将字符串转换成浮点数数值型

```
let x = parseFloat("129.13");
```

```
console.log(a);
```

var 与 let 的区别:

- var 可以重复定义同一变量而 let 不可以
- var 作用域在函数内, 没有块级作用域而 let 有块级作用域 (只在当前 {} 内有效)
- var 可以变量提升 (声明前访问不会报错, 值是 undefined) 而 let 不可以

例.

弹出输入框提示用户输入信息并将用户输入的内容输出到页面上, 控制台打印

```
<script>
```

```
let userName = prompt("请输入姓名");
```

```
document.write("<h3>您的输入为</h3>");
```

```
document.write("用户名: " + userName + "<br>"); // 允许 html 标签
```

```
console.log("用户名: ", userName);
```

```
</script>
```

例. 用户输入数字, 若数字小于10, 则在前面补0; 否则, 不补.

<script>

prompt 返回的是字符串

let inputNumber = prompt("请输入一个数字");

let num = Number(inputNumber);

let result;

if (num < 10 && num >= 0)

{
result = "0" + num;

}

else

result = num.toString();

document.write (处理后的结果是: {result});

反引号作定界符

</script>

声明常量

模版字符串 \${} 允许在字符串中插入变量或表达式。
其中的换行会直接被浏览器识别。浏览器会自动将 \${} 中的内容取值或计算。

当某个变量永远不会改变值时不用像:

替换成对应的字符串填充到模版中。

const x = 10;

let str = "第1行\n第2行";
console.log(str);

== 相等运算符

=== 严格相等运算符

允许检测的2个操作数类型转换

恒等/全等 不允许类型转换

5 == "5" "5" → 5 true

5 === "5" false

true == 1 true → 1 true

true === 1 false

null == undefined true

null === undefined false

数组

let array = [数据1, 数据2, ..., 数据n];

或 let array = new Array('a', 'b', 'c');

例. let num = [];

num[0] = 1;

num[1] = 2;

console.log(num);

例. 输入3个数据: 商品名称、数量、单价, 计算总价, 打印输出表格

订单确认

商品名称	商品数量	商品单价	商品总价
abc	13	10元	130元

```
<!DOCTYPE html>
<html lang="zh-CN">
<head meta="UTF-8">
<title> 例子 </title>
<style>
  table {
    margin: 20px 0;
    border-collapse: collapse; // 合并边框
  }
  th, td {
    border: 1px solid #000;
    text-align: center; // 水平居中
    vertical-align: middle; // 垂直居中
  }
</style>
</head>

<body>
  <script>
    let name = prompt("商品名称:");
    let number = Number(prompt("商品数量:"));
    let price = Number(prompt("商品单价(元):"));
    let totalPrice = price * number;
    document.write(`
      <table>
        <caption> 订单确认 </caption>
        <tr>
          <th> 商品名称 </th>
          <th> 商品数量 </th>
          <th> 商品单价 </th>
          <th> 商品总价 </th>
        </tr>
        <tr>
          <td> ${name} </td>
          <td> ${number} </td>
          <td> ${price} </td>
          <td> ${totalPrice} </td>
        </tr>
      </table>
    `);
  </script>
</body>
</html>
```

(可换成 height = line-height)

操作数组

新增

① push

数组.push(元素1, 元素2, ...)

元素添加到数组末尾, 返回该数组的新长度

② unshift

数组.unshift(元素1, 元素2, ...)

② shift

数组.shift()

删除第一个元素, 返回该元素的值

③ splice

数组.splice(起始位置, 删除个数)

删除指定元素

函数

① 声明函数

调用

function 函数名() {

函数名();

}

前缀一般是动词, 驼峰命名法

② 声明带参数的函数

function 函数名(参数列表) {

}

例如.

function getSum(num1, num2) {

document.write(num1 + num2);

getSum(9, 10);

数组作函数参数?

function getSum(arr) {

let sum = 0;

for (let i = 0; i < arr.length; i++)

sum += arr[i];

document.write(sum);

}

getSum([1, 2, 3, 4, 5, 6, 7, 8]);

③ 匿名函数

```
function ([参数列表]) {  
    ...  
}
```

这是没有名字的函数,无法直接使用. 需通过以下2种方式:

⇒ 函数表达式

将匿名函数赋值给一个变量并通过变量名称调用

(注意函数表达式的调用必须在其声明之后)

```
let fn = function (...) {  
    ...  
};  
fn(...);
```

例如

```
let fn = function (num1, num2) {  
    console.log(num1 + num2);  
}  
fn(9, 10);
```

// 声明

// 调用

⇒ 立即执行函数

将匿名函数的声明与调用放在一起

```
(function (...) {  
    ...  
})(...);
```

例如

```
(function (num1, num2) {  
    console.log(num1 + num2);  
})(9, 10);
```

对象

用于保存网站的用户信息, 比如姓名、年龄、电话号码等

是 JavaScript 中的一种数据类型, 可理解为一种无序数据集合

包括 { 静态特征 用变量表示
动态行为 用函数表示

类型 { 属性
方法

对象声明:

let 对象名 = { }

let 对象名 = new Object(); 例如

let 对象名 = {
属性名: 属性值,
方法名: 函数
}

let person = { }

let student = new Object();

let teacher = {
name: '张三',
gender: '女',
age: 28
}

构造函数

前面创建对象的方式一次只能创建一个对象

构造函数可以将对象中相同的属性和方法抽取出来封装到函数中

构造函数名首字母必须大写

声明:

function 构造函数名 (...){
当前对象 this. 属性 = 值;
this. 方法 = function (...){ }
}

调用:

new 构造函数名 (...);
调用构造函数必须使用 new

构造函数无需 return 就可返回结果

创建对象亦称为对象的实例化

最终目的: 得到能直接访问属性、调用方法的具体对象

方式一: 字面量实例化

let phone = {
brand: '苹果',
price: 1999;
callPhone: function(){
console.log('打电话');
};
};
console.log(phone.brand); // 访问实例的属性
phone.callPhone(); // 调用实例的方法

方式二: 构造函数实例化

function Phone (brand, price){
this.brand = brand;
this.price = price;
this.callPhone = function(){
console.log('打电话');
};
}

实例

let myphone = new Phone ('苹果', 11999);
console.log(myphone.brand);
myphone.callPhone();

例. 声明一个产品对象, 包括如下信息:

商品对象名称: goods1

商品名字: itsname

商品编号: num

商品毛重: weight

商品产地: address

页面显示:

商品名称: 苹果14 Pro Max

商品编号: 00001

商品毛重: 1kg

商品产地: 中国大陆

<script>

方法一

```
let goods1 = {  
  itsname: "苹果14 Pro Max",  
  num: "00001",  
  weight: "1kg",  
  address: "中国大陆"  
};  
document.write("商品名称:" + goods1.name + "<br>");  
document.write("商品名称:" + goods1['name'] + "<br>");  
document.write("商品编号:" + goods1.num + "<br>");  
goods1.weight = "10.00kg"; // 修改  
document.write("商品毛重:" + goods1.weight + "<br>");  
document.write("商品产地:" + goods1.address + "<br>");  
goods1.price = 10000; // 新增  
document.write("商品价格:" + goods1.price + "<br>");
```

方法二

```
function Goods ( itsname, num, weight, address ) {  
  this.itsname = itsname; this.num = num; this.weight = weight; this.address = address;  
  this.showInfo() {  
    document.write(`商品名称: ${this.itsname}<br>`);  
    document.write(`商品编号: ${this.num}<br>`);  
    document.write(`商品毛重: ${this.weight}<br>`);  
    document.write(`商品产地: ${this.address}<br>`);  
  }  
}
```

```
let goods2 = new Goods (" 苹果17", "1000", "1.0kg", "奥地利");  
goods2.showInfo();
```

属性名

```
for ( let k in goods1 ) {  
  document.write(k);  
  document.write(goods1[k]);  
  console.log(k);  
  console.log(goods1[k]);  
}
```

属性值

每次循环k被赋值为对象的一个属性名

goods1.k 就错了! undefined.

k = 'itsname'
k = 'num'
k = 'weight'
k = 'address'

</script>

JavaScript 有 3 种对象

- 自定义对象
- 内置对象 (JS 自带)
- 浏览器对象 (JS 独有)

内置对象

例如 Math, Date, Array, String, ...

Math 是一个内置对象，有一些数学常数属性和数学函数方法，它不是一个函数对象

Math 用于 Number 类型而不支持 BigInt

Math 与其它全局对象不同，它不是一个构造器，它的所有方法与属性都是静态的

Math.PI, Math.sin(x), ...

Math 的常量使用 JavaScript 的全精度浮点数来定义

Math 不是构造函数，不需要使用 new 来调用

随机函数

Math.random()

返回一个 0-1 之间，包含 0，不包含 1 的随机小数 [0, 1)

如何生成 A~B 区间内的随机整数？

Math.floor(Math.random() * (B - A + 1) + A)
向下取整

例. 生成随机颜色

```
let arr = ['red', 'pink', 'blue', 'green'];
```

```
let index = Math.floor(Math.random() * arr.length);
```

```
document.write = (
```

```
  <div class="box" style="width:100px; height:100px;
```

```
    background-color: #${arr[index]} "> 加宽高或加文字可显示
```

```
  </div>
```

```
);
```

CSS 中最后一条的分号可以省略

Date 对象

日期对象是一个构造函数, 必须使用 new 来创建

不加 new 直接调用 Date() 就只会返回个字符串而不是对象, 无法调用 Date 对象的方法去操作时间

创建

```
let d1 = new Date();  
document.write(d1);  
document.write("<br>");
```

方式1: 创建当前系统时间的 Date 对象

```
let d2 = new Date("2025-9-23");  
document.write(d2);  
document.write("<br>");
```

→ "September 23, 2025 11:35:18"

方式2: 通过字符串创建指定日期

```
let d3 = new Date(2025, 9, 23);  
document.write(d3);  
document.write("<br>");
```

→ 2025, 9, 23, 11, 35, 18

方式3: 通过数字参数创建指定日期

返回的是10月的信息

数字型参数返回的月份大1, 尽量用字符串型参数

日期格式化

```
let date = new Date();
```

```
let year = date.getFullYear();
```

→ 返回 0~11

```
let month = date.getMonth() + 1;
```

```
let day = date.getDate();
```

```
const weekMap = ["日", "-", "=", "三", "四", "五", "六"]; // 映射
```

```
let week = weekMap[date.getDay()];
```

→ 返回 0~6

```
document.write(`今天是${year}年${month}月${day}日 星期${week}`);
```

时分秒格式化

```
function getFormattedTime() {
```

```
    let date = new Date();
```

0~23

```
    let hours = date.getHours() < 10 ? "0" + date.getHours() : date.getHours();
```

0~59

```
    let minutes = date.getMinutes() < 10 ? "0" + date.getMinutes() : date.getMinutes();
```

0~59

```
    let seconds = date.getSeconds() < 10 ? "0" + date.getSeconds() : date.getSeconds();
```

```
    return `${hours}:${minutes}:${seconds}`;
```

```
}
```

```
document.write(getFormattedTime());
```

时间戳 (Date 总的毫秒数)

```
let time1 = new Date();  
console.log (time1.valueOf());  
console.log (time1.getTime());  
let time2 = (+) new Date();  
console.log (time2);  
let time3 = Date.now(); H5 新增方法  
console.log (time3);
```

倒计时

要用时间戳

用户输入时间的总毫秒数 - 现在时间的总毫秒数 = 剩余时间的总毫秒数,
再将剩余时间的秒数转换为天、时、分、秒.

function getTime (time) { ← 用户输入的时间

```
    let nowTime = + new Date();
```

```
    let inputTime = + new Date (time);
```

```
    let leftTime = (inputTime - nowTime) / 1000;
```

```
    let day = parseInt (leftTime / 60 / 60 / 24); 天
```

```
    let hour = parseInt (leftTime / 60 / 60 % 24); 小时
```

```
    let minute = parseInt (leftTime / 60 % 60); 分钟
```

```
    let second = parseInt (leftTime % 60); 秒
```

```
    return day + '天' + hour + '小时' + minute + '分钟' + second + '秒';  
}
```

```
console.log (getTime ('2025-9-23 11:35:18'));
```

引用类型 (复杂数据类型)

通过 new 关键字创建的对象. 例如 Object, Array, Date ...

引用类型变量里存放地址而真正的对象实例存放堆空间

```
let num1 = 10;
let num2 = num1;
num2 = 20;
console.log(num1); // 10
```

指向

```
let obj1 = {
  age: 18
};
let obj2 = obj1;
obj2.age = 20;
console.log(obj1.age); // 20
```

const, let, var 之间,

const 优先选择, 语义化更好

若是基础类型, 绑定的是“值” → 值不能改.

若是引用类型, 绑定的是“地址” → 地址不能改, 但地址指向的内容可以改

相当于 "Number(...)"

```
let num1 = +prompt('输入第一个数字:'); // 把字符串转成数字
```

```
let num2 = +prompt('输入第二个数字:');
console.log(num1 + num2);
```

let 可改 const

```
for (let i = 0; i < arr.length; i++)
{
  console.log(arr[i]);
}
```

let 不可改 const

```
let arr = ['red', 'pink'];
arr.push('yellow');
```

```
let obj = {
  username: '张三',
  age: 18
};
```

let 可以改 const

```
obj.gender = '女'; // 追加内容
```

```
let arr = [];
arr = [1, 2, 3];
```

let 不可以改 const

```
let obj = {};
```

```
obj = {
  age: 18,
  gender: '男'
};
```

重新绑定了地址.

原来是
“空”

DOM (Document Object Model)

功能: 通过操作 html 文档中的内容, 实现网页特效和用户交互

DOM 对象

浏览器根据 html 元素生成的 JS 对象

功能: 所有标签属性都可以在这个对象上找到, 修改这个对象的属性会自动映射到 html 标签上

```
<script>
  const div = document.querySelector('div');
  div.innerText = '修改后的文字';
  div.style.color = 'red';
  div.className = 'new-class';
</script>
```

获取 DOM 元素

主要是根据 "CSS 选择器获取 DOM 元素" 的方法

1) 获取匹配到的第一个元素

```
const div = document.querySelector('.box'); // 关于 class
const nav = document.querySelector('#nav'); // 关于 id
const first = document.querySelector('ul li'); // 关于 li
```

返回的是一个 HTML Element 对象, 没有匹配到就返回 null

2) 获取匹配到的所有元素

```
const li = document.querySelectorAll('ul li');
const div = document.querySelectorAll('div');
console.log(div);
```

返回的是 NodeList 对象集合, 是一个 "伪数组"

伪数组是有长度和索引号的数组, 但是 没有 pop, push 这些方法
要想得到里面的每一个对象, 需要用 for 遍历

例如

```
const div = document.querySelectorAll('div');
for (let i = 0; i < div.length; i++)
{
  console.log(div[i]);
}
```

伪数组

其他方式:

⇒ 依据 id 获取

```
document.getElementById('nav');
```

返回匹配的 DOM 元素对象

⇒ 依据标签名获取

```
document.getElementsByTagName('div');
```

返回 HTMLCollection (伪数组)

⇒ 依据类名获取

```
document.getElementsByClassName('box');
```

返回 HTMLCollection (伪数组)

对象. innerText 属性

将文本内容添加/更新到指定位置

只显示纯文本而不解析标签

```
const one = document.querySelector('.one');
one.innerText = '这是第一个div中的内容';
one.innerText = '<strong>加粗</strong>';
```

例. 随机抽奖

<body>

<h1>随机抽奖</h1>

<h2>一等奖: ??</h2>

<h3>二等奖: ??</h3>

<h4>三等奖: ??</h4>

<script>

```
const nameList = ["小张", "小李", "小王"];
```

```
const one = document.querySelector('.one');
const two = document.querySelector('.two');
const three = document.querySelector('.three');
```

获取DOM元素

(对应3个奖项的span)

```
function Draw(list, count) {
  const result = [];
  const copyList = [...list];
  for (let i = 0; i < count; i++) {
    const index = Math.floor(Math.random * copyList.length);
    result.push(copyList[index]);
    copyList.splice(index, 1);
  }
  return result;
}
```

```
const winners = Draw(nameList, 3);
```

```
one.innerHTML = winners[0];
```

```
two.innerHTML = winners[1];
```

```
three.innerHTML = winners[2];
```

</script>

</body>

对象. innerHTML 属性

将文本内容添加/更新到指定位置

解析标签, 多标签可使用模板字符串

```
const two = document.querySelector('.two');
two.innerHTML = `
  <div class="box">
    <h3>标题</h3>
    <p>内容</p>
    <ul>
      <li>列表1</li>
      <li>列表2</li>
    </ul>
  </div> `;
```

操作元素属性

对象.属性 = 值

例. 随机显示图片

```
<body>
  

  <script>
    const imgCount=3;
    const index= Math.floor((Math.random()*imgCount));
    const img= document.querySelector('img');
    或 const img= document.getElementById('randomImg');
    img.src = `./images/${index}.png`;
  </script>
</body>
```

通过style修改样式属性

对象.style.样式属性 = 值

```
const box = document.querySelector('.box');
box.style.backgroundColor = 'yellow';
```

注意遇到带短横线的属性使用驼峰命名

通过className修改样式属性

对象.className = 类名

替换:

```
const div = document.querySelector('div');
div.className = 'box';
```

通过classList操作样式属性

```
const div = document.querySelector('div');
```

追加

```
div.classList.add('类名');
```

删除

```
div.classList.remove('类名');
```

切换(有就删,
没有就加上)

```
div.classList.toggle('类名');
```

例、通过DOM完成以下颜色设置

- 测试1
- 测试2
- 测试3

```
<body>
```

```
<ul>
```

```
<li id="test1"> 测试1</li>
```

```
<li id="test2"> 测试2</li>
```

```
<li id="test3"> 测试3</li>
```

```
</ul>
```

```
<script>
```

```
const test1 = document.getElementById('test1');
```

```
const test2 = document.getElementById('test2');
```

```
const test3 = document.getElementById('test3');
```

```
test1.style.color = 'red';
```

```
test2.style.color = 'blue';
```

```
test3.style.color = 'green';
```

```
</script>
```

```
</body>
```

操作表单属性

表单对象.属性 = 值

username.value = 'admin'; // 注意: 表单的值用的是 value 而不是 innerHTML
password.type = 'text';

全选反选

爱好: ☐ 全选 ☐ 唱歌 ☐ 跳舞 ☐ 画画

提交

<body>

<form action=" " >

<p>

爱 好

<input type="checkbox" class="all"> 全选

<input type="checkbox" class="hobby"> 唱歌

<input type="checkbox" class="hobby"> 跳舞

<input type="checkbox" class="hobby"> 画画

</p>

</form>

<script>

const all = document.querySelector('.all');

const hobby = document.querySelectorAll('.hobby');

伪数组

all.addEventListener('click', function(){
 for(let i=0; i<hobby.length; i++){
 hobby[i].checked = this.checked;
 }
 updateBtnStatus();
});

for(let i=0; i<hobby.length; i++){

hobby[i].addEventListener('click', function(){

let num = document.querySelectorAll('.hobby:checked').length;

if(num == hobby.length)
 all.checked = true;

else
 all.checked = false;

updateBtnStatus();
});

自定义属性

</script>

不是html 标签自带的属性, 而是开发人员自己添加的

命名规范:

须以 "data-" 开头且一个标签可以有多个自定义属性

<div data-id="1" data-abc="xyz">1</div>

自定义属性

操作方法:

对象.dataset. 自定义属性

const div = document.querySelector('div'); console.log(div.dataset.id);

<p>

<input type="submit" value="提交">

</p>

const sbt =

document.querySelector('input[type=submit]');

"全选" sbt.disabled = true;

"控制
所有爱好"

根据爱好框的勾选数量

控制

全选框的状态

function updateBtnStatus(){

let num = document.querySelectorAll('.hobby:checked').length;

if(num == 0)

sbt.disabled = true;

else

sbt.disabled = false;

}

定时器

间歇函数 (每隔一段时间自动执行一段代码而不需要人为触发)

开启定时器

关闭定时器

`clearInterval` (变量名)

`setInterval` (函数名, 间隔时间)

一个程序通常开多个定时器, 注意: 没有()!
返回定时器编号

作用: 每隔一段时间执行一次函数

间隔时间单位是毫秒

```
let i=10;
function fn(){
  document.write(`${i}`);
  i--;
}
setInterval(fn, 1000);
```

```
let i=10;
let n;
function fn(){
  document.write(`${i}`);
  if(i==0)
    clearInterval(n);
  else i--;
}
n = setInterval(fn, 1000);
```

定时器中

可以用匿名函数:

```
setInterval(function(){
  console.log('第一名');
}, 1000);
```

例. 阅读注册协议, 60s后按钮才可使用

```
const btn = document.querySelector('button');
let i = 60;
btn.disabled = true;
function fn(){
  btn.innerHTML = '已阅读用户协议('+i+' )';
  if(i==0){
    clearInterval(timer); // 关闭定时器
    btn.disabled = false;
  }
  i--
}
let timer = setInterval(fn, 1000); // 开启定时器
```

事件监听 (绑定)

事件: 系统内部发生的动作或事情

事件监听: 程序检测是否有事件发生, 一旦发生就立即调用一个函数作为响应

亦称为注册事件或绑定事件

元素对象. addEventListener (事件类型, 要执行的函数) 方式1

三要素:

- 事件源: 哪个 DOM 元素被事件触发了
- 事件类型: 用何方式触发, 比如点击 click 或 鼠标经过 mouseover
- 事件调用的函数: 做什么事

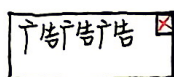
方式一

```
<button>按钮</button>
<script>
  const btn = document.querySelector('button');
  btn.addEventListener('click', function() {
    alert('我被点击啦!');
  });
</script>
```

每次“点击”都会执行

这里的函数可以是函数名或函数表达式的变量名

例. 关闭广告



```
<script>
  const closeBtn = document.querySelector('.close');
  const adv = document.querySelector('.adv');
  closeBtn.addEventListener('click', function() {
    adv.style.display = 'none';
  });
</script>
```

5秒后可关闭

```
setTimeout(function() {
  closeBtn.style.display = 'block';
}, 5000);
```

仅执行一次 setTimeout (回调函数, 等待的毫秒数)

功能: 延时一段时间执行回调函数

清除: `let timer = setTimeout(函数, 时间)`
`clearTimeout(timer)`

鼠标事件 (由鼠标触发)

click 鼠标点击
mouseenter 鼠标经过
mouseleave 鼠标离开

焦点事件 (表单获得光标)

focus 获得焦点
blur 失去焦点

键盘事件 (键盘触发)

keydown 按下触发
keyup 抬起触发

文本事件 (表单输入触发)

input 用户输入

例. 随机点名

小宝

开始 结束

```
<div class = 'box'>
  <div class = 'name'></div>
  <div class = 'button'>
    <button class = 'start'>开始</button>
    <button class = 'close' disabled>结束</button>
  </div>
</div>
```

<script>

```
const uname = ['小张', '小王', '小李'];
```

```
const nameElm = document.querySelector('.name');
```

```
const startBtn = document.querySelector('.start');
```

```
const closeBtn = document.querySelector('.close');
```

```
let timer = null;
```

```
startBtn.addEventListener('click', function() {
```

```
  startBtn.disabled = true;
```

```
  closeBtn.disabled = false.
```

```
  timer = setInterval(function() {
```

```
    const index = Math.floor((Math.random() * uname.length));
```

```
    nameElm.innerHTML = uname[index];
```

或 textContent

```
  }, 50);
```

```
});
```

```
closeBtn.addEventListener('click', function() {
```

```
  startBtn.disabled = false;
```

```
  closeBtn.disabled = true;
```

```
  clearInterval(timer);
```

放前面

```
});
```

</script>

素框

文本框获得焦点后显示下拉列表, 同时将文本框边框变成黄色, 失去焦点后隐藏下拉列表
另外, 添加计时器每秒自动更新一次文本框中的内容

注意 input 的样式: `border: 0;` // 去除边框
`outline: 0;` // 去除焦点边框
"placeholder"

<script>

```
const arr = ['全部商品', '红米', '空调', '电视', '洗衣机', '充电宝', '手机'];
```

```
const ipt = document.querySelector('input');
```

```
const search = document.querySelector('.search');
```

```
const ul = document.querySelector('ul');
```

```
ul.style.display = 'none';
```

```
let i = 0;
```

```
function fn() {
```

```
    ipt.placeholder = arr[i];
```

```
    if (i == arr.length - 1)
```

```
        i = 0;
```

```
    else
```

```
        i++;
```

```
}
```

```
setInterval(fn, 1000);
```

```
ipt.addEventListener('focus', function() {
```

```
    search.style.borderColor = 'orange';
```

```
    ul.style.display = 'block';
```

```
}
```

```
ipt.addEventListener('blur', function() {
```

```
    search.style.borderColor = 'grey';
```

```
    ul.style.display = 'none';
```

```
})
```

</script>

事件对象

事件对象：存储了事件触发时的相关信息，比如点在哪个位置或按下哪个键

获取事件对象：事件绑定的函数中第一个参数就是事件对象，一般命名为 event, ev, e

```
元素.addEventListener('事件', function(e){  
  })
```

例如.

```
const btn = document.querySelector('button');  
btn.addEventListener('click', function(e){  
  console.log(e);  
})
```

常见属性

```
const div = document.querySelector('div');  
div.addEventListener('click', function(e){  
  console.log('当前事件类型:', e.type);  
  鼠标/光标  
  console.log('相对浏览器窗口左上角的位置: X=' + e.clientX + ', Y=' + e.clientY);  
  console.log('相对当前元素(触发事件的DOM)左上角的位置: X=' + e.offsetX +  
    ', Y=' + e.offsetY);  
  console.log('按下的键:', e.key);  
})
```

评论

(1) 评论字数统计: 用户输入文字, 统计用户输入的字数

判断事件: 输入事件 input

不断获取文本框中字符串的长度: 文本域, value, length

把获取到的数字显示到下面

(2) 按下回车键或点击按钮发表评论

```
<script>
```

```
const text = document.querySelector('textarea');
```

```
const btn = document.querySelector('button');
```

```
const cnt = document.querySelector('count');
```

```
const cmt = document.querySelector('comment');
```

```
text.addEventListener('input', function() {
```

```
    cnt.innerHTML = text.value.length + '/200字';
```

```
});
```

// 回车键

```
text.addEventListener('keyup', function(e) {
```

```
    if (e.key === 'Enter') {
```

```
        if (text.value.trim() !== '') {
```

```
            cmt.innerHTML += `<div>${text.value.trim()}</div>`;
```

```
            text.value = '';
```

```
            cnt.innerHTML = text.value.length + '/200字';
```

```
        }
```

```
    }
```

```
});
```

// 按钮

```
btn.addEventListener('click', function(e) {
```

```
    if (text.value.trim() !== '') {
```

```
        cmt.innerHTML += `<div>${text.value.trim()}</div>`;
```

```
        text.value = '';
```

```
        cnt.innerHTML = text.value.length + '/200字';
```

```
    }
```

```
});
```

```
cmt.innerHTML = `<div>${text.value.trim()}</div>` + cmt.innerHTML;
```

// 上下位置就反过来了

```
</script>
```

环境对象 this

每个函数中都有环境对象 this

普通函数中 this 指向 window (浏览器窗口) 对象

```
例如. function fn() {  
    console.log(this);  
}
```

↓
window

其他函数中 this 指向调用者

```
const btn = document.querySelector('button');  
btn.addEventListener('click', function() {  
    console.log(this);  
});
```

↓
btn, 即 <button style="background-color: orange;">
点我 </button> 这个元素.

回调函数

当函数作为参数传递给别的函数时, 这个函数就是回调函数

```
let n = setInterval(fn, 1000);
```

```
btn.addEventListener('click', function() {});
```

解绑事件 (匿名函数无法解绑!)

removeEventListener (事件类型, 事件处理函数)

```
const btn = document.querySelector('button');
```

```
function fn() {  
    alert('我被点击了!');  
}
```

```
btn.addEventListener('click', fn);
```

```
btn.addEventListener('click', fn);
```

Tab 栏切换

```
<body>
  <div class="tab">
    <div class="tab-list">
      <ul>
        <li class="current">商品介绍</li>
        <li>规格与包装</li>
        <li>售后保障</li>
        <li>商品评价(50000)</li>
        <li>手机社区</li>
      </ul>
    </div>
    <div class="tab-con">
      <div class="item" style="display: block;">
        商品介绍模块内容
      </div>
      <div class="item">
        规格与包装模块内容
      </div>
      <div class="item">
        售后保障模块内容
      </div>
      <div class="item">
        商品评价(50000)模块内容
      </div>
      <div class="item">
        手机社区模块内容
      </div>
    </div>
  </div>
  <script>
    const lis = document.querySelectorAll('ul li');
    const divs = document.querySelectorAll('div item');
    for (let i = 0; i < lis.length; i++) {
      lis[i].addEventListener('mouseenter', function() {
        document.querySelector('.current').classList.remove('current');
        lis[i].classList.add('current');
        for (let j = 0; j < divs.length; j++) {
          divs[j].style.display = 'none';
        }
        divs[i].style.display = 'block';
      });
    }
  </script>
```

页面滚动事件

应用场景: 页面滚动到某区域后, 固定导航栏, 返回顶部等

事件名: scroll

功能:

监听整个页面的滚动, 若监听某个元素的内部滚动, 则直接给元素加监听事件

```
window.addEventListener('scroll', function() {  
  })
```

scrollLeft 属性: 往左滚出去看不见的距离

scrollTop 属性: 往上滚出去看不见的距离

可读写
读: let 变量 = 元素.scrollTop
写: 元素.scrollTop = 数值

① 监听页面滚动:

```
window.addEventListener('scroll', function() {  
  const n = document.documentElement.scrollTop; // HTML元素  
  console.log(n);  
});
```

② 监听元素内部滚动:

```
div.addEventListener('scroll', function() {  
  console.log(this.scrollTop);  
});
```

例. 滚动页面 (大于300 像素时显示侧边栏, 点击“返回顶部”可返回顶部)

“不显示”的2种方法: ① display: none;

② opacity: 0; // 透明度设为0, 完全透明, 不显示

<script>

```
const aside = document.querySelector('.aside');  
const rtn = document.querySelector('.return');  
window.addEventListener('scroll', function() {
```

```
  const n = document.documentElement.scrollTop; // 读  
  if (n > 300) {  
    aside.style.opacity = 1;  
  }  
  else  
    aside.style.opacity = 0;  
}
```

```
});
```

```
rtn.addEventListener('click', function() {  
  document.documentElement.scrollTop = 0; // 写
```

```
});
```

</script>

查找DOM节点

① 获取父节点

```
const son1 = document.querySelector('.son1');
```

```
son1.parentNode.innerHTML = '789';
```

② 获取子节点

```
const father = document.querySelector('.father');
```

```
father.children[0].innerHTML = '789';
```

添加DOM节点

① 创建节点(元素)

```
document.createElement('标签名');
```

② 若想在界面中看到, 还需将节点添加到父元素中

父元素.appendChild(要插入的元素);

默认插入到父元素中的最后一个子元素

若想插入到父元素中某节点之前:

父元素.insertBefore(要插入的元素, 指定节点)

例. `<div class="father"> </div>`

`<script>`

```
const father = document.querySelector('.father');
```

```
const son = document.createElement('div');
```

```
son.innerHTML = '123';
```

```
father.appendChild(son);
```

```
const son2 = document.createElement('div');
```

```
son2.innerHTML = '456';
```

```
father.insertBefore(son2, father.children[0]);
```

`</script>`

删除节点

父元素.removeChild(要删除的元素)

```
father.removeChild(son2);
```

例. 轮播图

<script>

```
const images = ['./img/0.jpg', './img/1.jpg', './img/2.jpg', './img/3.jpg', './img/4.jpg', './img/5.jpg'];
```

```
const box = document.querySelector('.box');
```

```
const img = document.querySelector('.picture');
```

```
const ul = document.querySelector('.slide');
```

```
let i = 0;
```

```
function fn() {
```

```
  i++;
```

```
  if (i === images.length)
```

```
    i = 0;
```

```
  img.src = images[i];
```

```
  updateDot();
```

```
}
```

```
function updateDot() {
```

```
  document.querySelector('.slide.active').classList.remove('active');
```

```
  ul.children[i].classList.add('active');
```

```
}
```

```
let timer = setInterval(fn, 1000); // 自动播放功能
```

```
const left = document.querySelector('.left');
```

```
const right = document.querySelector('.right');
```

} 获取左右箭头

```
box.addEventListener('mouseenter', function() {
```

```
  clearInterval(timer);
```

```
  left.style.display = 'block';
```

```
  right.style.display = 'block';
```

```
});
```

} 鼠标移到图片上

```
box.addEventListener('mouseleave', function() {
```

```
  timer = setInterval(fn, 1000);
```

```
  left.style.display = 'none';
```

```
  right.style.display = 'none';
```

```
});
```

} 鼠标移出图片

```
left.addEventListener('click', function() {
```

```
  i--;
```

```
  if (i === -1)
```

```
    i = images.length - 1;
```

```
  img.src = images[i];
```

```
  updateDot();
```

```
});
```

} 点击左箭头

```
right.addEventListener('click', fn); // 点击右箭头
```

```
for (let j = 0; j < ul.children.length; j++) {
```

```
  ul.children[j].addEventListener('click', function() {
```

```
    i = j;
```

```
    img.src = images[i];
```

```
  });
```

```
  updateDot();
```

```
</script>
```

} 点击小圆点

| | | | |
|------|-----------|------|-------|
| 123 | '123' | == | === |
| | | true | false |
| null | undefined | true | false |
| true | 1 | true | false |
| 0 | '' | true | false |